
LemmaPortfolio: Shared-Budget Lemma Selection for Multiple Lean Goals

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 A small set of lemmas may support several proofs, but useful lemmas can also
2 be redundant. We introduce LEMMAPORTFOLIO to study how models select lem-
3 mas under a budget shared across multiple goals. Given statements alone, models
4 choose three of sixteen Lean declarations for eight target theorems. The bench-
5 mark contains 30 development and 60 test episodes drawn from MATHLIB and fil-
6 tered against fixed selectors. Coverage is based on Lean-checked direct references
7 in existing proofs; it measures historical use, not proof sufficiency. We exhaus-
8 tively score all 560 possible selections per episode, allowing every answer to be
9 compared with the exact optimum. Across seven CLI/API runs, the highest exact-
10 optimal count is 30/60, and the highest mean coverage is 5.58 targets against an
11 attainable 6.25. An audit using the hidden dependency labels clarifies what the
12 task measures: on 53/60 episodes, an optimal selection exists among triples that
13 maximize total individual use frequency, yet 184/246 non-optimal model answers
14 fall outside those sets. LEMMAPORTFOLIO thus provides an exactly scored bench-
15 mark for shared-budget lemma selection. Its results show a gap between covering
16 many targets and choosing an optimal portfolio, while its substantial frequency-tie
17 structure prevents interpreting that gap as overlap reasoning alone.

18 1 Introduction

19 Selecting lemmas across several goals combines two judgments: where each lemma is likely to be
20 used, and how its applications overlap with those of other lemmas. Choosing three individually
21 useful results need not yield the most useful collection. This shared-budget decision motivates
22 LEMMAPORTFOLIO, a retrospective task that tests use recognition and complementary selection
23 together, not overlap reasoning in isolation.

24 Each episode presents eight target statements and sixteen candidate declarations from one MATHLIB
25 module. The model selects three candidates without proofs or dependency annotations. A target
26 counts as covered if its stored proof directly references a selected candidate. For a mathematical
27 agent, this models a limited part of proof planning: allocating a shared lemma budget across pending
28 goals. We evaluate that selection decision, not an agent’s ability to complete the proofs.

29 Premise-selection systems usually serve one proof state or theorem. LeanDojo and Magnushammer
30 retrieve Lean and Isabelle premises, respectively [Yang et al., 2023, Mikula et al., 2024]. LeanHam-
31 mer evaluates selection through automated proving [Zhu et al., 2026], while LeanSearch v2 retrieves
32 premise groups for individual theorems [Gao et al., 2026]. Graph-based lemma mining accounts for
33 reuse and redundancy with known dependencies [Kaliszyk and Urban, 2015]. UseFor learns theo-
34 rem generation from subsequent reuse and proof-likelihood signals [Kasriel et al., 2025]. We instead
35 ask how one selection budget should be shared across targets when the dependency annotations are
36 withheld.

Our contribution is a controlled benchmark and an auditable evaluation of joint selection, with exact optima and retrospective analyses of overlap and portfolio distance.

2 The lemma portfolio task

Each episode has a candidate set C of size sixteen and a target set T of size eight. Candidates are Lean declarations, mostly theorems but also definitions or constructors. Candidate and target types are displayed under episode-local IDs C01–C16 and T01–T08. For candidate c , let $S_c \subseteq T$ contain the targets whose stored proof terms directly reference c . References are extracted from elaborated terms without following dependencies into c ’s own body. A portfolio $P \subseteq C$, with $|P| = 3$, has coverage

$$f(P) = \left| \bigcup_{c \in P} S_c \right|, \quad f^* = \max_{Q \subseteq C, |Q|=3} f(Q).$$

With the sets S_c known, this is a maximum-coverage problem, a standard monotone submodular objective [Nemhauser et al., 1978]. Here those sets are hidden from the model. We enumerate all $\binom{16}{3} = 560$ portfolios to compute f^* and accept every maximizer. The primary metric is the number of episodes receiving an exactly optimal portfolio; mean target coverage is secondary. Missing or malformed answers receive zero for both metrics and remain in the denominator.

Candidate	Target ID (T prefix omitted)							
	01	02	03	04	05	06	07	08
C05	•							•
C08				•				•
C13			•			•		
C16					•		•	

C05, C08, C16: 5 targets.
C05, C13, C16: 6 targets.
All four candidates cover two targets individually. Replacing C08 loses T04, retains T08 through C05, and gains T02 and T06.

Figure 1: Overlap matters in the biproduct episode MLP4B_0003. Dots mark historical direct references, shown here for four of the sixteen candidates. Both original Pro and Sol xhigh answers selected the first portfolio; the second attains the episode optimum.

What a candidate statement says. For the mathematical content behind Figure 1, consider C13. In a category with zero morphisms, let $(f_j)_{j \in J}$ admit a biproduct, and let $p_j : f_j \rightarrow X$ be a family of morphisms. The candidate states

$$\iota_j \gg \text{desc}(p) = p_j,$$

where ι_j is the biproduct inclusion, $\text{desc}(p)$ is the induced map to X , and $\gg$ denotes composition with the left map first. Target T06 states that inclusion into a subfamily’s biproduct followed by its canonical map into the full biproduct equals the corresponding full-family inclusion. Its stored proof uses C13. These are mathematical abbreviations; the prompts retain the full Lean binders, qualified names, and typeclass assumptions.

3 Construction and audit

We mine MATHLIB v4.33.0 [The mathlib Community, 2020, 2026], using disjoint modules and statements for development and test. Candidates and targets within an episode come from the same module. To avoid instances dominated by one declaration, no candidate may cover more than four targets. At least seven candidates must occur in the target proofs, with at least four of these absent from every optimal portfolio. We also require overlapping uses, multiple one-target-short portfolios, and at least two target occurrences for each member of an optimum. Remaining thresholds and pinned-Lean checks are specified in the supplement.

We discard an episode if any of twenty fixed selectors attains optimal coverage: eighteen syntactic rules and two rules using true annotations, namely occurrence-frequency ranking and greedy marginal coverage. All ties favor the earlier candidate ID. This conditions the dataset on the failure of these particular rules; it does not establish general resistance to greedy optimization. Section 6 quantifies the resulting tie-order sensitivity.

The maximum coverage is six on 45 test episodes and seven on the other 15, giving an attainable mean of 6.25. A post-hoc masking audit finds residual source-name prefixes in 6/60 test and 2/30 development episodes. Excluding the six affected test episodes leaves every run at or below 27/54 exact optima (Appendix E). This is an exclusion sensitivity check, not evidence that the remaining statements were absent from training.

4 Evaluation and results

Table 1 reports seven post-release runs on the same sixty episodes. Each uses twelve released prompts, containing five episodes apiece, submitted separately without tools or proof-use labels. Astra and Sol use Codex CLI; Fable uses the Anthropic Messages API. Fable 5.1 receives additional benchmark context after an earlier refusal-only attempt; the archive distinguishes these conditions and identifies missing refusal records. All seven runs return sixty valid portfolios. Six preliminary web-interface response sets, including all missing and malformed answers, appear in Appendix C.

Both Astra and Sol have two xhigh repeats. Planned third repeats were cancelled after completed scores were known. Test labels were known to the experimenter but not supplied to the models. Different interfaces and settings prevent a compute-matched model ranking. Baselines are the exact uniform-random expectation and a character TF-IDF selector (Appendix A).

Table 1: Direct selection on sixty test episodes; every model run returns sixty valid portfolios. Intervals are nominal Wilson 95% intervals for the exact-optimal rate. Coverage counts targets, not completed proofs. The oracle uses hidden annotations; random values are expectations. *Additional benchmark context.

Model / effort / run	Exact /60	95% CI (%)	Coverage /8
Oracle (true annotations)	60	—	6.25
Uniform random	0.14	—	2.35
Character TF-IDF	0	0.0–6.0	3.77
<i>Post-release: Codex CLI</i>			
GPT-6 Astra / xhigh / 1	27	33.1–57.5	5.47
GPT-6 Astra / xhigh / 2	30	37.7–62.3	5.53
GPT-5.6 Sol / xhigh / 1	20	22.7–45.9	5.03
GPT-5.6 Sol / xhigh / 2	15	15.8–37.2	5.00
GPT-6 Astra / max / 1	30	37.7–62.3	5.58
<i>Post-release: Anthropic Messages API</i>			
Claude Fable 5 / xhigh	26	31.6–55.9	5.52
Claude Fable 5.1 / xhigh*	26	31.6–55.9	5.58

Every run exceeds both statement-only baselines on exact count and mean coverage, but none is optimal on more than half the episodes. Astra max and context-conditioned Fable 5.1 each cover 335 target occurrences out of the attainable 375 (89.3%), yet their exact counts differ: 30/60 and 26/60. Astra xhigh repeats attain 27 and 30 optima; Sol xhigh repeats attain 20 and 15. These differences are comparable to several between-model gaps. The nominal intervals assume independent episode outcomes; five-episode prompts may induce dependence, and the intervals do not quantify run-to-run variability (Appendix A).

Frequency ties and error patterns. An optimum maximizes summed individual use counts on 53/60 episodes. The frequency-maximizing sets contain 6.8 triples on average (median 4; range 1–35). With true counts known, overlap-aware tie resolution thus suffices on most, but not all, episodes. Among 246 non-optimal follow-up answers, 184 (74.8%) lie outside these sets; 52 select a suboptimal tied triple where the set contains an optimum, and 10 select from a set containing none. This partition does not identify usage-estimation versus overlap errors: the models’ estimates are unobserved, and seven episodes require departing from frequency maximization. Moreover, 71 errors admit higher coverage at the same multiset of individual use counts, including 19 of the 184 outside-set errors (Appendix D).

104 **Near-optimal coverage and portfolio distance.** Across the seven follow-up runs, 163 answers
105 are one target short of optimal. Of these, 26 require two candidate replacements and three require re-
106 placing all three candidates; the other 134 admit a single replacement. These are answer instances on
107 the same sixty episodes, not independent problems. Distances use true annotations retrospectively;
108 no model repair is evaluated (Appendix A).

109 5 Target-wise prediction

110 An exploratory pipeline predicts up to four candidate occurrences per target, retains the first two,
111 and maximizes predicted coverage over all 560 triples, breaking ties by candidate ID. Direct versus
112 pipeline exact counts are 4/15 versus 1/15 for original Pro, 1/20 versus 0/20 for original Sol,
113 and 8/20 versus 4/20 for fresh Astra xhigh calls. A true optimum lies among Astra’s predicted
114 maximizers on 13/20 episodes, a label-assisted upper bound rather than achieved performance. The
115 pipeline does not improve these paired subsets, but prediction errors, truncation, and tie-breaking
116 are confounded; the result does not establish that decomposition is ineffective (Appendix F).

117 6 Limitations

118 The construction filters create a substantial tie-order artifact: true-label greedy with construction-
119 order ties loses on all 60 test episodes, but reversing the order solves 57/60, and some greedy tie
120 path is optimal on 59/60. This is a deliberately filtered selection task, not evidence that greedy
121 coverage is generally inadequate. An unfiltered control sample and a dense-retriever baseline are
122 planned; neither is included here.

123 Historical direct use is neither logical necessity nor proof sufficiency. Alternative proofs may use
124 different declarations, and elaborated terms include references inserted by Lean. We do not follow
125 transitive dependencies. Fixed candidate pools and equal costs restrict the planning analogy. Resid-
126 ual name prefixes and possible training exposure remain concerns. Sixty episodes, heterogeneous
127 conditions, and limited repeats do not support fine-grained model rankings or broad claims about
128 mathematical ability. Human-expert comparisons and downstream prover tests are still needed to
129 establish practical value.

130 7 Conclusion

131 LEMMAPORTFOLIO tests a shared-budget decision relevant to proof planning. On 53/60 episodes,
132 true use counts restrict the search for an optimum to frequency ties. Yet 184/246 non-optimal follow-
133 up answers fall outside the frequency-maximizing sets; 52 choose a suboptimal member when an
134 optimum is available there, and 10 occur where no such member is optimal. This distinguishes
135 observable selection patterns, not their causes: inaccurate use estimates and overlap mistakes can
136 coexist. A dense-retriever baseline and unfiltered controls should test how much difficulty comes
137 from recognizing use rather than resolving ties. Downstream prover experiments must establish
138 whether better selections help complete proofs.

References

- Guoxiong Gao, Zeming Sun, Jiedong Jiang, Yutong Wang, Jingda Xu, Peihao Wu, Bryan Dai, and Bin Dong. LeanSearch v2: Global premise retrieval for Lean 4 theorem proving, 2026. URL <https://arxiv.org/abs/2605.13137>.
- Cezary Kaliszyk and Josef Urban. Learning-assisted theorem proving with millions of lemmas. *Journal of Symbolic Computation*, 69:109–128, 2015. doi: 10.1016/j.jsc.2014.09.032. URL <https://doi.org/10.1016/j.jsc.2014.09.032>.
- Timothe Kasriel, Thomas Lu, Devon Ding, Jingxuan He, and Dawn Song. Usefulness-driven learning of formal mathematics. In *NeurIPS Workshop on Mathematical Reasoning and AI*, 2025. URL <https://openreview.net/forum?id=SHeRsXGRcZ>.
- Mathlib Initiative. Mathlib constant dependencies. Hugging Face dataset, 2026a. URL <https://huggingface.co/datasets/mathlib-initiative/mathlib-const-dep>. Version 4.33.0; exact 128-shard inventory recorded in the artifact.
- Mathlib Initiative. Mathlib types. Hugging Face dataset, 2026b. URL <https://huggingface.co/datasets/mathlib-initiative/mathlib-types>. Version 4.33.0; exact 128-shard inventory recorded in the artifact.
- Maciej Mikula, Szymon Tworkowski, Szymon Antoniak, Bartosz Piotrowski, Albert Q. Jiang, Jin Peng Zhou, Christian Szegedy, Łukasz Kuciński, Piotr Miłoś, and Yuhuai Wu. Magnushammer: A transformer-based approach to premise selection. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=oYjPk8mqAV>.
- George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming*, 14:265–294, 1978. doi: 10.1007/BF01588971.
- The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*. ACM, 2020. doi: 10.1145/3372885.3373824.
- The mathlib Community. mathlib4 source snapshot for version 4.33.0. Git repository commit, 2026. URL <https://github.com/leanprover-community/mathlib4>. Commit db584cd6d46c92f209a44c0f1c829460d327499d.
- Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22:209–212, 1927.
- Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023. doi: 10.52202/075280-0944. URL <https://arxiv.org/abs/2306.15626>.
- Thomas Zhu, Joshua Clune, Jeremy Avigad, Albert Qiaochu Jiang, and Sean Welleck. Premise selection for a Lean hammer. In *International Conference on Learning Representations*, 2026. doi: 10.48550/arXiv.2506.07477. URL <https://openreview.net/forum?id=m04JJNeRK6>.

Technical supplement

A Scoring and baselines

The primary score counts episodes with $f(P) = f^*$. Mean coverage averages $f(P)$ over episodes. The archive also reports the mean of $f(P)/f^*$ and occurrence recall, the fraction of recorded candidate–target pairs whose candidate is selected. Invalid answers contribute zero and remain in all denominators. The aggregate ratio 335/375 in the main text is a ratio of coverage totals, not the mean of per-episode ratios.

For k exact optima in $n = 60$ episodes, we report the two-sided Wilson score interval without continuity correction [Wilson, 1927], with $\hat{p} = k/n$ and $z = \Phi^{-1}(0.975)$:

$$\frac{\hat{p} + z^2/(2n) \pm z\sqrt{\hat{p}(1 - \hat{p})/n + z^2/(4n^2)}}{1 + z^2/n}.$$

These are nominal binomial intervals, not confidence statements about unfiltered MATHLIB or future deployments. They approximate episodes as independent despite joint five-episode prompts, and do not include run-to-run variability. Repeats are reported separately, not pooled or selected by their best score. We make no paired significance claim from overlapping or non-overlapping intervals. Oracle and random entries are exact quantities on the fixed benchmark, not sampled success proportions.

For a target referenced by d_t displayed candidates, uniform selection of three distinct candidates gives

$$\Pr(t \text{ is covered}) = 1 - \frac{\binom{16-d_t}{3}}{\binom{16}{3}}.$$

Summing over targets and averaging over episodes yields expected mean coverage. The expected exact-optimal count is $\sum_e |\mathcal{O}_e|/560$, where $\mathcal{O}_e$ is the complete optimum set for episode e . These are analytic expectations, not sampled runs.

The TF-IDF baseline uses character 3–5-grams from the twenty-four displayed statements. A nonzero gram count $n(g, s)$ receives weight $(1 + \ln n(g, s))(1 + \ln(25/(1 + \text{df}(g))))$, followed by L2 normalization. Each target contributes its largest cosine similarity to a selected candidate. We maximize the sum over all 560 portfolios, breaking ties by candidate-ID order. This baseline was added after construction and is not one of the twenty filters.

The distance of a submitted portfolio P to optimality is

$$d(P, \mathcal{O}) = \min_{Q \in \mathcal{O}} (3 - |P \cap Q|).$$

We enumerate its 39 distinct single-replacement neighbors and count those in $\mathcal{O}$. Of the 134 one-target-short answers admitting a single replacement, 122 have one optimal neighbor, ten have two, and two have three. The remaining 29 have none. The reported distances concern the 163 one-target-short answers, excluding already optimal responses. No repair strategy is evaluated.

B Construction and Lean verification

In addition to Section 3, each episode must have $f^* \geq 6$, at most 32 optimal portfolios, at least two targets with multiple displayed dependencies, and at least two portfolios of coverage $f^* - 1$. Mean pairwise Jaccard similarity of distinct tokens in target statements is at most 0.6. The complete filters and mining receipts are preserved in the archive.

The pinned-Lean audit checks all 720 target terms: 240 development rows with 386 candidate–target occurrences and 480 test rows with 814 occurrences match the stored annotations. A separate expression-equivalence check finds no matching statement expressions within an episode; it does not establish mathematical inequivalence. Four biproduct targets are reassocated variants already generated by MATHLIB, so routine corollaries are part of the task.

True-label greedy with the construction tie order is one target short on every test episode. Reversing that order attains 57/60 optima; some greedy tie path attains an optimum in 59/60 episodes. Uniformly breaking each greedy tie gives an expected 37.41 optima. These procedures have access to hidden labels and are construction diagnostics, not model baselines.

C Preliminary web-interface responses

Table 2 retains the six original web-interface response sets; with the seven CLI/API runs, all thirteen sets and their recorded conditions remain in the archive. Names are interface labels, not independently authenticated serving identities. Pro has five missing answers and Qwen 3.8 five malformed answers, all in D12; these remain in the sixty-episode denominator. Original run order, hidden system prompts, and feedback are not fully recoverable. These preliminary records are not pooled with the CLI/API runs.

Table 2: Preliminary web-interface selections on the same sixty test episodes. Intervals follow Appendix A; invalid or missing responses score zero.

Recorded model label	Exact /60	95% CI (%)	Coverage /8	Valid /60
GPT SOL 5.6 Pro	25	30.1–54.3	4.98	55
GPT SOL 5.6 xhigh	18	19.9–42.5	5.08	60
DeepSeek Instant + DeepThink	1	0.3–8.9	4.20	60
DeepSeek Expert + DeepThink	7	5.8–22.2	4.60	60
Qwen 3.8 Max–Thinking	12	11.8–31.8	4.32	55
Qwen 3.7 Plus–Thinking	6	4.7–20.1	4.05	60

D Frequency, greedy selection, and overlap

We audit all 780 direct answer slots, excluding 10 invalid or missing answers from pattern comparisons but retaining them in score denominators. Of 770 valid answers, 527 are non-optimal. The main-text subset comprises seven follow-up runs: 420 valid answers and 246 non-optimal instances on the same sixty episodes.

Let $d(c) = |S_c|$. A frequency-maximizing triple maximizes $\sum_{c \in P} d(c)$, without discounting overlap. The fixed rule ranks by $d(c)$ with ascending candidate-ID ties; the tie-aware rule accepts every maximizing triple. Sequential greedy is different: at each of three steps it maximizes additional union coverage given earlier selections. We check both ascending-ID ties and every possible greedy tie path. Enumeration of all 560 triples and an independent permutation check agree on which triples are greedy-reachable.

Across the sixty test episodes, frequency-maximizing sets contain 408 triples in total: mean 6.8, median 4, and range 1–35 per episode. At least one optimum belongs to these sets on 53/60 episodes (88.3%); 71/78 distinct optimal triples are frequency-maximizing. The seven exceptions are 0000, 0005, 0022, 0027, 0035, 0050, and 0051, each prefixed MLP4B_. This audit weights episodes equally, not by model success. Independent enumeration at the third-largest count threshold reproduces every tied set.

The fixed frequency and greedy matches are necessarily non-optimal because the test set excludes their successes. Allowing all frequency ties matches 230/420 follow-up answers overall, including 168/174 optimal answers and 62/246 non-optimal answers. Among errors, the fixed frequency count is 21/246 (8.5%), rising to 62/246 (25.2%) with all ties; the corresponding greedy counts are 19/246 and 86/246. Of the 62 frequency-maximizing errors, 52 select a suboptimal member of a set that contains an optimum; the other 10 occur on the seven exceptional episodes. The 184 errors outside the frequency-maximizing sets comprise 151 on the other 53 episodes and 33 on the seven exceptions. Six successful answers also depart from frequency maximization, all on exceptional episodes. Departures therefore cannot simply be labeled usage-estimation errors.

For a separate overlap check, let $h(P)$ be the sorted multiset of individual use counts in P . Comparing triples with the same profile finds higher coverage for 71/246 follow-up errors (28.9%): 52 inside and 19 outside the frequency-maximizing sets. At a fixed profile, a larger union implies less repeated coverage. The checks intersect; neither gives a causal partition or an upper bound on all overlap mistakes. Models may use inaccurate frequency estimates, but those estimates are unobserved. No repair is tested. Per-answer comparisons, tie sets, input hashes, and intervals are reproduced by `verify_selection_patterns.py`.

Table 3: Post-hoc patterns among valid non-optimal answers. Freq. ranks by true occurrence counts; greedy uses true marginal coverage. Fixed uses ascending-ID ties; any allows all ties. Profile counts answers admitting greater coverage at the same multiset of individual counts. Columns overlap and are not error categories.

Run	Errors	Freq.		Greedy		Profile
		Fixed	Any	Fixed	Any	
GPT SOL 5.6 Pro	30	1	8	5	12	11
GPT SOL 5.6 xhigh	42	2	8	4	11	10
DeepSeek Instant + DeepThink	59	2	10	1	6	20
DeepSeek Expert + DeepThink	53	0	11	2	10	15
Qwen 3.8 Max-Thinking	43	2	3	2	7	8
Qwen 3.7 Plus-Thinking	54	1	2	1	4	8
GPT-6 Astra / xhigh / 1	33	3	9	4	13	9
GPT-6 Astra / xhigh / 2	30	3	6	3	11	9
GPT-5.6 Sol / xhigh / 1	40	2	6	3	9	10
GPT-5.6 Sol / xhigh / 2	45	2	10	3	13	11
GPT-6 Astra / max / 1	30	3	11	3	14	11
Claude Fable 5 / xhigh	34	3	10	1	12	10
Claude Fable 5.1 / xhigh / context	34	5	10	2	14	11
Follow-up total	246	21	62	19	86	71
All thirteen response sets	527	29	104	34	136	143

E Masking-exclusion sensitivity

The source-name-prefix scan flags test IDs 0012, 0033, 0037, 0042, 0046, and 0059 (each prefixed MLP4B_). It finds 33 occurrences involving 24 source declarations in these six episodes. We exclude those IDs and re-score all thirteen saved direct-response sets (Table 4); the original sixty-episode results remain unchanged. This extends the archived original-panel check without new inference or corrected prompts. No run exceeds 27/54 exact optima. The exclusion changes episode composition, so it neither estimates a causal masking effect nor establishes absence of training exposure.

Table 4: Post-hoc sensitivity after excluding six episodes with residual name prefixes. Invalid answers remain in the denominator. The oracle uses true labels; random values are expectations. Context denotes the additional Fable 5.1 system prompt.

Model / baseline	Exact /54	Coverage /8	Valid /54
Oracle (true annotations)	54	6.26	–
Uniform random	0.12	2.34	–
Character TF-IDF	0	3.70	54
GPT SOL 5.6 Pro	22	5.07	50
GPT SOL 5.6 xhigh	15	5.13	54
DeepSeek Instant + DeepThink	1	4.26	54
DeepSeek Expert + DeepThink	7	4.67	54
Qwen 3.8 Max-Thinking	10	4.41	50
Qwen 3.7 Plus-Thinking	3	3.98	54
GPT-6 Astra / xhigh / 1	23	5.44	54
GPT-6 Astra / xhigh / 2	27	5.54	54
GPT-5.6 Sol / xhigh / 1	17	5.06	54
GPT-5.6 Sol / xhigh / 2	13	5.07	54
GPT-6 Astra / max / 1	27	5.59	54
Claude Fable 5 / xhigh	22	5.52	54
Claude Fable 5.1 / xhigh / context	23	5.61	54

F Target-wise widths and ties

The original paired subset uses D04, D05, D06, and D12, not the protocol’s optional subset. Pro lacks direct D12 answers, leaving fifteen complete pairs. Original run order and feedback are unrecoverable. Astra uses fresh direct and target-wise calls on these twenty episodes, separate from its full runs.

Direct versus target-wise exact counts are 4/15 versus 1/15 for original Pro and 1/20 versus 0/20 for original Sol. Fresh Astra xhigh calls attain 8/20 versus 4/20, with mean coverage 5.30 versus 5.00. These are paired subset comparisons, not additional full-benchmark runs.

Keeping $q = 1, 2, 3, 4$ predictions per target gives Astra exact counts 2, 4, 4, 4 and mean coverages 4.55, 5.00, 4.80, 4.80: offline rescoring of the same predictions. At $q = 2$, a true optimum lies among predicted maximizers on 13/20 Astra episodes, versus four selected. The original fixed twenty-episode Pro and Sol diagnostics each have a 9/20 upper bound, versus achieved counts 1/20 and 0/20. These upper bounds require labels to resolve ties.

G Evaluation records and reproducibility

Original model names are the recorded web-interface labels. Astra and Sol follow-up records report Codex CLI 0.153.0, fresh processes, blank working directories, and no tools; temperature and numeric reasoning budgets were not exposed. The first Astra xhigh run preceded the protocol freeze. Exact CLI invocation files are unavailable. Both Fable runs use the streaming Anthropic Messages API, anthropic==1.4.0, xhigh effort, and a 128,000-token output limit.

The Fable 5.1 context-conditioned run follows twelve reported bare-prompt refusals absent from the supplied export. Retained small classifier checks are not substitutes for those records or scored answers. The system context is preserved verbatim. Fable 5 resumed after a transport overload, retaining completed answers without mathematical repair.

Run `python3 -B verify_all.py` to check hashes and scores offline, without authenticating serving identities or rerunning inference. Annotation extraction requires pinned Lean 4.33.0 and MATHLIB; mining additionally requires 256 versioned Parquet shards [Mathlib Initiative, 2026a,b]. The archive documents versions, commands, and evidence boundaries.

Author responsibility and assistance. The authors conceived and directed the study. Codex agents assisted with filtering choices, implementation, response transcription, and manuscript drafting and editing. The authors are responsible for the scientific claims, interpretation, and accompanying material.